Indian Institute of Technology Gandhinagar

Department of Computer Science & Engineering

CS 432 – Databases

Track-1 · Assignment 1 · Module B

GateGuard

Entry Gate Management System

Conceptual Design Report

Team Name: Lumos

Team Members

Name	Roll Number
Pushkar Nikunjkumar Modi	25120035
Chintan Bhikhabhai Akhaja	25120027
Krina Paghadar	24110169
Divisht	24110112
Akshata Pawar	25120026

Instructor: Prof. Yogesh K. Meena

Contents

1 Introduction . . . **4**
- 1.1 Problem Statement . . . 4
- 1.2 Proposed Solution . . . 4
- 1.3 Project Scope . . . 4
- 1.4 Assignment Objectives . . . 5
- 1.5 Core Functionalities . . . 5

2 UML Diagrams . . . **6**
- 2.1 UML Class Diagram . . . 6
  - 2.1.1 Class Diagram Key Features . . . 6
  - 2.1.2 Classes Summary . . . 7
  - 2.1.3 Relationships in UML Class Diagram . . . 7
- 2.2 Sequence Diagram . . . 8
  - 2.2.1 Sequence Diagram Description . . . 11
- 2.3 Use Case Diagram . . . 12
  - 2.3.1 Actors and Use Cases . . . 12
- 2.4 Activity Diagram . . . 13
  - 2.4.1 Activity Diagram Description . . . 15

3 Entity-Relationship (ER) Diagram . . . **16**
- 3.1 Complete ER Diagram . . . 16
- 3.2 ER Diagram Components . . . 16
  - 3.2.1 Entity Classification . . . 16
  - 3.2.2 Attribute Types in the ER Diagram . . . 16
  - 3.2.3 Relationship Types . . . 18
  - 3.2.4 Weak Entity . . . 18

4 UML to ER Transition . . . **19**
- 4.1 Classes to Entities Conversion . . . 19
- 4.2 Associations to Relationships Conversion . . . 20
- 4.3 Key Definitions . . . 20
  - 4.3.1 Primary Keys . . . 20
  - 4.3.2 Foreign Keys . . . 21
- 4.4 Multiplicity Adjustments . . . 21
- 4.5 Participation Constraints . . . 22

5 Relationship Justifications . . . **23**
- 5.1 R1: MemberType → Member (categorizes) . . . 23
- 5.2 R2: VehicleType → Vehicle (classifies) . . . 23
- 5.3 R3: Member → Vehicle (owns) . . . 24
- 5.4 R4: Member → PersonVisit (makes) . . . 25
- 5.5 R5: Vehicle → PersonVisit (used_in) . . . 25
- 5.6 R6: Vehicle → VehicleVisit (makes v_visit) . . . 26
- 5.7 R7: Gate → PersonVisit – Entry (entry) . . . 27
- 5.8 R8: Gate → PersonVisit – Exit (exit) . . . 27
- 5.9 R9: Gate → VehicleVisit – Entry (v_entry) . . . 28
- 5.10 R10: Gate → VehicleVisit – Exit (v_exit) . . . 29
- 5.11 R11: Gate → GateOccupancy (tracks) – Identifying . . . 29

5.12 R12: Role → User (assigned_to) . . . 30
5.13 Relationship Summary . . . 31

6 Additional Constraints . . . 32
6.1 CHECK Constraints . . . 32
6.1.1 Temporal Constraints (Visit Tables) . . . 32
6.1.2 Exit Gate Consistency . . . 32
6.1.3 Occupancy Count . . . 32
6.1.4 Email Format Validation . . . 32
6.1.5 Age Range Validation . . . 33
6.1.6 License Plate Format . . . 33
6.1.7 Username and Password Constraints . . . 33
6.1.8 Type Name Enumeration Constraints . . . 33
6.2 UNIQUE Constraints . . . 34
6.3 NOT NULL Constraints . . . 34
6.4 Referential Integrity Summary . . . 35
6.5 Indexes . . . 35

7 Database Schema Overview . . . 37
7.1 Schema Statistics . . . 37
7.2 Sample Data Distribution . . . 37
7.3 Normalization Analysis . . . 37

8 Team Member Contributions . . . 39
8.1 Collaborative Efforts . . . 39
8.2 Tools and Technologies . . . 40

9 Conclusion . . . 41
9.1 Summary of Deliverables . . . 41
9.2 Key Design Strengths . . . 41

A Complete SQL Schema . . . 43

List of Figures

1 GateGuard – UML Class Diagram (Complete System) 6
2 GateGuard – Sequence Diagram (Entry/Exit) 8
3 GateGuard – Use Case Diagram 12
4 GateGuard – Activity Diagram (Entry and Exit Flow) 13
5 GateGuard – Complete ER Diagram (Crow's Foot Notation with TikZ) . 17

List of Tables

1 GateGuard Core Functionalities 5
2 UML Classes and Their Attributes 7
3 UML Associations and Multiplicity 7
4 Actors and Their Use Cases 12
5 Entity Classification and Colour Coding 16
6 Attribute Type Notation 16
7 Relationship Types and Notation 18
8 UML Class to ER Entity Mapping 19
9 UML Association to ER Relationship Mapping 20
10 Primary Key Mapping 21
11 Foreign Key Mapping (12 Relationships) 21
12 UML Multiplicity to ER Cardinality Mapping 21
13 Participation Constraints in the ER Diagram 22
14 Complete Relationship Summary (12 Foreign Keys) 31
15 UNIQUE Constraints 34
16 NOT NULL Column Counts per Table 34
17 Referential Integrity Actions 35
18 Database Indexes 36
19 GateGuard Database Statistics 37
20 Sample Data Record Counts 37
21 Team Member Contributions 39
22 Tools and Technologies Used 40

1. Introduction

1.1 Problem Statement

In campuses, residential complexes, and corporate facilities, entry and exit management is often handled by security personnel using handwritten logs or verbal verification. This manual process is slow, error-prone, and vulnerable to security breaches, as unauthorised visitors or vehicles can enter without proper tracking.

During peak hours, long queues form at gates, causing congestion and increasing the risk of missed or incorrect entries. Additionally, authorities lack real-time visibility into who is on the premises, making it difficult to respond to emergencies or monitor occupancy levels effectively.

1.2 Proposed Solution

GateGuard is an Entry Gate Management System designed specifically for the IIT Gandhinagar campus to address these challenges. The system provides:

- **Automated Entry/Exit Tracking** – Recording every person and vehicle entry/exit with timestamps and gate identification.
- **Real-time Occupancy Monitoring** – Maintaining live occupancy counts at each gate for congestion analysis.
- **Role-Based Access Control** – Implementing a Guard/Admin/SuperAdmin hierarchy for system security.
- **Comprehensive Visitor Tracking** – Categorising members as Residents, Students, or Visitors with distinct access policies.
- **Vehicle Registration & Verification** – Linking vehicles to owners and tracking vehicle movements independently.
- **Multi-Gate Synchronisation** – Supporting different entry and exit gates for the same visit, reflecting real campus operations.
- **Audit Trail** – Maintaining `CreatedAt`/`UpdatedAt` timestamps on all records for accountability.

1.3 Project Scope

The GateGuard system is built on a robust relational database architecture comprising **10 interconnected entities**, organised into four functional layers:

1. **Member Management** – `MemberType`, `Member`
2. **Vehicle Management** – `VehicleType`, `Vehicle`
3. **Gate Control** – `Gate`, `GateOccupancy`
4. **Security & Users** – `Role`, `User`
5. **Visit Tracking** – `PersonVisit`, `VehicleVisit`

The database follows **Third Normal Form (3NF)** to ensure data integrity, eliminate redundancy, and maintain consistency. With **12 foreign key relationships**, **30+ constraints**, and **105 sample records**, the system demonstrates a production-ready design.

1.4 Assignment Objectives

This Module B report demonstrates:

1. Comprehensive UML class diagram design with 10 classes and 12 associations.
2. Additional behavioural UML diagrams (Sequence, Activity) modelling system workflows.
3. Detailed Entity-Relationship (ER) diagram following standard notation.
4. Systematic UML-to-ER transition methodology.
5. Justification of all 12 relationships with real-world examples from sample data.
6. Documentation of all constraints (CHECK, UNIQUE, FK, NOT NULL).

1.5 Core Functionalities

The system supports **seven core functionalities**, exceeding the minimum requirement of five:

Table 1: GateGuard Core Functionalities

#	Functionality	Description
F1	Member Registration	Register new members with categorisation (Resident/Student/Visitor)
F2	Vehicle Registration	Register vehicles with type classification and optional owner linkage
F3	Person Entry Tracking	Record person entry with gate, timestamp, and optional vehicle
F4	Person Exit Tracking	Record person exit with different gate support and duration calculation
F5	Vehicle Entry/Exit Tracking	Independent vehicle movement tracking through gates
F6	Occupancy Monitoring	Real-time gate-wise occupancy count management
F7	User Authentication	Role-based system access for Guards, Admins, and SuperAdmins

2. UML Diagrams

UML (Unified Modeling Language) diagrams provide a high-level conceptual design of the GateGuard system. Following the methodology described in *Elmasri & Navathe, Chapter 10* [1], we model the system using multiple UML diagram types to capture both structural and behavioural aspects.

2.1 UML Class Diagram

The UML class diagram (Figure 1) presents the complete structural design of GateGuard, showing all 10 entity classes organised into five packages: *Member Management*, *Vehicle Management*, *Gate Control*, *Security & Users*, and *Visit Tracking*.

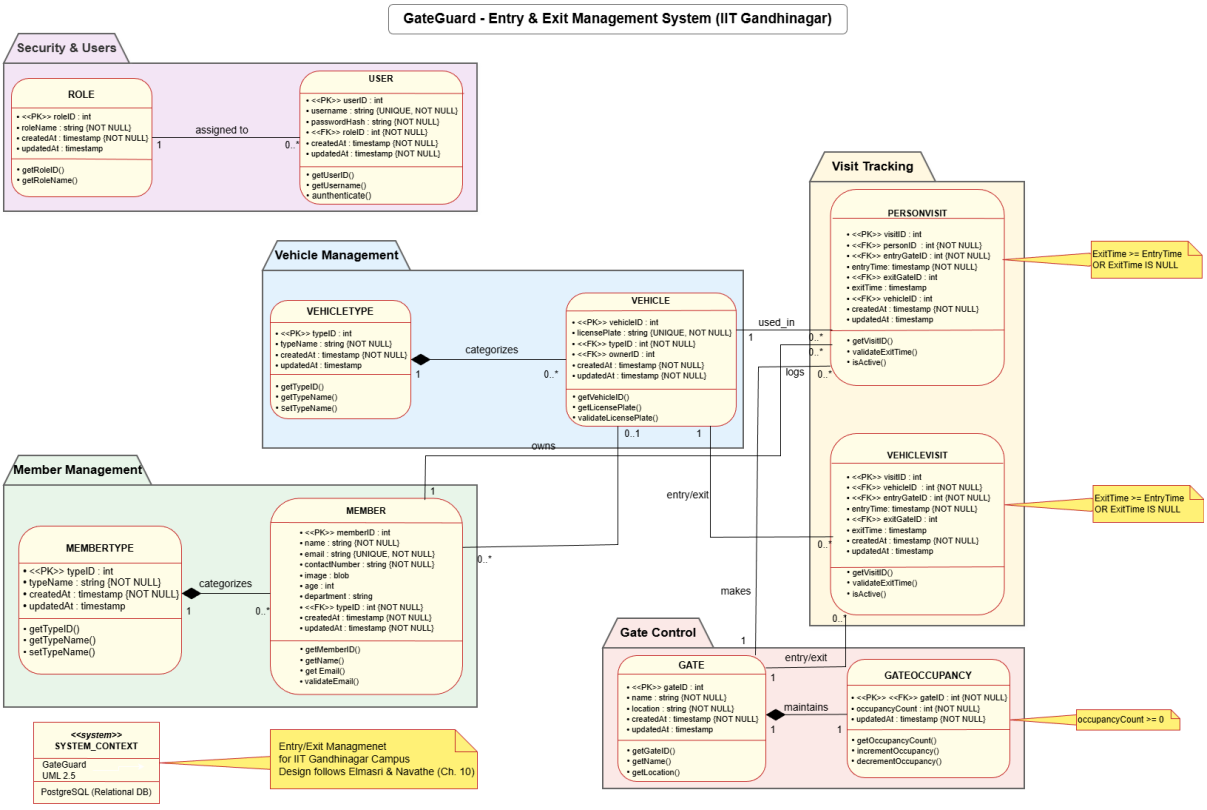

Figure 1: GateGuard – UML Class Diagram (Complete System)

2.1.1 Class Diagram Key Features

- **10 Classes** across 5 packages, each with attributes, methods, and stereotype annotations.
- **Stereotypes:** «PK» for Primary Keys, «FK» for Foreign Keys.
- **Visibility:** All attributes are private (-), all methods are public (+).
- **Constraints:** Annotated directly on attributes: `{NOT NULL}`, `{UNIQUE, NOT NULL}`.
- **Relationships:** 10 associations shown with multiplicity labels.
- **Composition (♦):** `MemberType`–`Member`, `VehicleType`–`Vehicle`, `Gate`–`GateOccupancy`.
- **Association (–):** All other relationships use standard association lines.

2.1.2 Classes Summary

Table 2: UML Classes and Their Attributes

Package	Class	Attrs	Methods	Key Attributes
Member Mgmt	MEMBERTYPE	4	3	typeID **PK**, typeName
Member Mgmt	MEMBER	10	4	memberID **PK**, email **UQ**, typeID **FK**
Vehicle Mgmt	VEHICLETYPE	4	3	typeID **PK**, typeName
Vehicle Mgmt	VEHICLE	6	3	vehicleID **PK**, licensePlate **UQ**, typeID **FK**, ownerID **FK**
Gate Control	GATE	5	3	gateID **PK**, name, location
Gate Control	GATEOCCUPANCY	3	3	gateID **PKFK**, occupancyCount
Security	ROLE	4	2	roleID **PK**, roleName
Security	USER	6	3	userID **PK**, username **UQ**, roleID **FK**
Visit Tracking	PERSONVISIT	9	3	visitID **PK**, personID **FK**, entryGateID **FK**
Visit Tracking	VEHICLEVISIT	8	3	visitID **PK**, vehicleID **FK**, entryGateID **FK**

2.1.3 Relationships in UML Class Diagram

Table 3: UML Associations and Multiplicity

#	Source	Target	Multiplicity	Label
R1	MEMBERTYPE	MEMBER	$1 \rightarrow 0..*$	categorizes
R2	VEHICLETYPE	VEHICLE	$1 \rightarrow 0..*$	categorizes
R3	GATE	GATEOCCUPANCY	$1 \rightarrow 1$	maintains
R4	MEMBER	PERSONVISIT	$1 \rightarrow 0..*$	makes
R5	MEMBER	VEHICLE	$0..* \rightarrow 0..1$	owns
R6	VEHICLE	PERSONVISIT	$1 \rightarrow 0..*$	used_in
R7	VEHICLE	VEHICLEVISIT	$1 \rightarrow 0..*$	logs
R8	GATE	PERSONVISIT	$1 \rightarrow 0..*$	entry / exit
R9	GATE	VEHICLEVISIT	$1 \rightarrow 0..*$	entry / exit
R10	ROLE	USER	$1 \rightarrow 0..*$	assigned_to

Note on Gate Relationships: In the UML diagram, Gate has a single association line to PersonVisit and VehicleVisit labeled "entry / exit". In the database implementation, this is decomposed into *two* separate foreign keys (`EntryGateID` and `ExitGateID`), yielding 12 total foreign key relationships.

2.2 Sequence Diagram

The sequence diagram (Figure 2) models the temporal interaction between system components during a complete entry and exit cycle. It shows message flow between actors (Guard), the terminal interface, and the database system.

Figure 2: GateGuard – Sequence Diagram (Entry/Exit)

2.2.1 Sequence Diagram Description

The sequence diagram captures two main workflows:

Entry Flow (17 actions):

1. Guard verifies ID card of approaching person.
2. Guard approves entry and enters Member ID into the terminal.
3. System checks if person has a vehicle (decision point).
4. If vehicle present: Guard enters license plate and vehicle type; system finds or creates vehicle record.
5. If no vehicle: System bypasses vehicle processing.
6. System executes three **parallel operations** (FORK):
   - Path 1: `INSERT INTO PersonVisit` (entry record)
   - Path 2: `INSERT INTO VehicleVisit` (conditional, if vehicle present)
   - Path 3: `UPDATE GateOccupancy SET OccupancyCount = OccupancyCount + 1`
7. System displays confirmation; Guard allows entry.

Exit Flow (10 actions):

1. Guard enters Member ID for exit at exit gate.
2. System finds active visit: `SELECT FROM PersonVisit WHERE ExitTime IS NULL`.
3. System executes three **parallel operations** (FORK):
   - Path 1: `UPDATE PersonVisit SET ExitTime, ExitGateID`
   - Path 2: `UPDATE VehicleVisit SET ExitTime, ExitGateID` (conditional)
   - Path 3: `UPDATE GateOccupancy SET OccupancyCount = OccupancyCount - 1`
4. System displays exit confirmation; Guard allows exit.

2.3 Use Case Diagram

The use case diagram (Figure 3) depicts the functional requirements from the perspective of different system actors and their interactions with GateGuard.

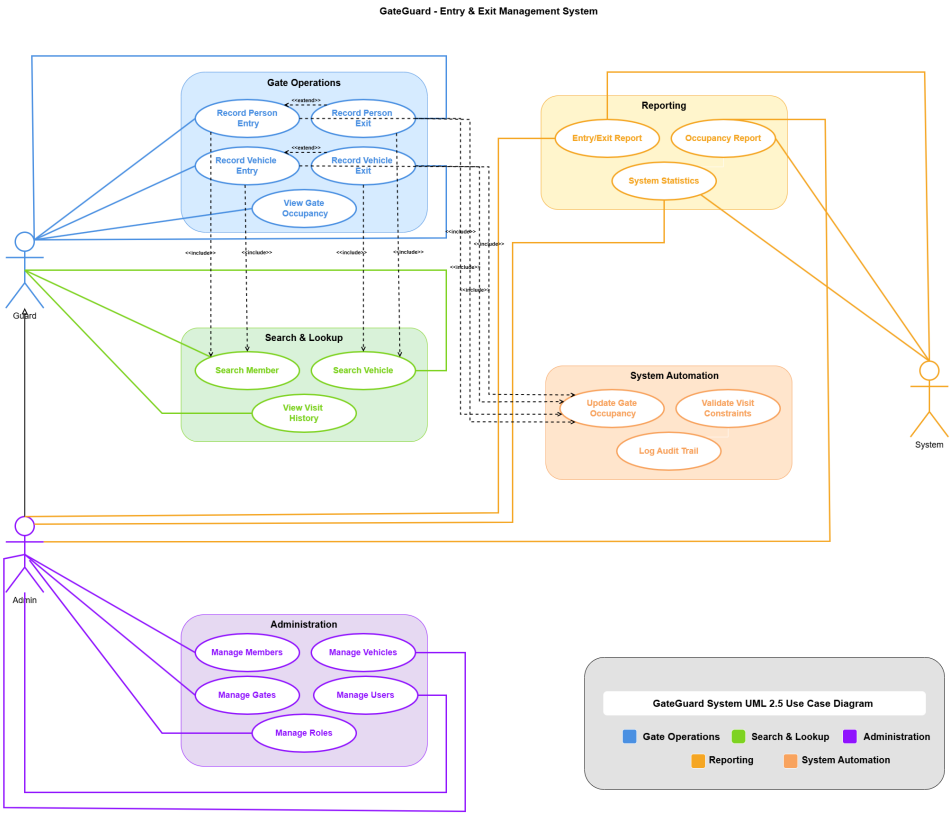

Figure 3: GateGuard – Use Case Diagram

2.3.1 Actors and Use Cases

Table 4: Actors and Their Use Cases

Actor	Use Cases
Guard	Verify ID, Record Entry, Record Exit, Check Vehicle, View Occupancy
Admin	Manage Members, Manage Vehicles, Manage Gates, View Reports
SuperAdmin	Manage Users, Manage Roles, System Configuration, Full Access
Member	Present ID at Gate, Register Vehicle

2.4 Activity Diagram

The activity diagram (Figure 4) models the workflow logic and decision points during gate entry and exit operations, including parallel processing paths.

Figure 4: GateGuard – Activity Diagram (Entry and Exit Flow)

2.4.1 Activity Diagram Description

The activity diagram models two swim-lane processes:

Entry Process:

- Begins with ID card verification by the Guard.
- **Decision Node:** "Has Vehicle?" – splits into vehicle and pedestrian paths.
- Vehicle path: Enter plate number → Find/Create vehicle in database → Decision: "Plate exists?" (reuse or create).
- **Merge Node:** All paths converge.
- **Fork Node:** Three parallel database operations execute simultaneously.
- **Join Node:** Synchronises all three operations before displaying confirmation.

Exit Process:

- Guard enters Member ID for exit at the exit gate terminal.
- System locates the active PersonVisit record (`WHERE ExitTime IS NULL`).
- **Fork Node:** Three parallel UPDATE operations (PersonVisit, VehicleVisit, GateOccupancy).
- **Join Node:** Synchronises all updates before displaying exit confirmation.

3. Entity-Relationship (ER) Diagram

The ER diagram provides a detailed database-level conceptual model of GateGuard, following standard notation as described in *Elmasri & Navathe, Chapter 7* [1] and *Silberschatz, Korth & Sudarshan, Chapter 6* [2].

3.1 Complete ER Diagram

The complete ER diagram is rendered using TikZ/LaTeX and is presented as a full-page landscape figure on the following page for maximum clarity.

3.2 ER Diagram Components

3.2.1 Entity Classification

The 10 entities are organised into four functional categories, each colour-coded in the diagram:

Table 5: Entity Classification and Colour Coding

Category	Colour	Entities	Purpose
Master Data	Yellow	MemberType, VehicleType, Role	Reference/lookup tables
Core Entities	Blue	Member, Vehicle, Gate	Primary domain entities
Transactional	Green	PersonVisit, VehicleVisit, GateOccupancy	Event/state tracking
System	Purple	User	Security and access control

3.2.2 Attribute Types in the ER Diagram

The diagram uses standard ER notation to distinguish attribute types:

Table 6: Attribute Type Notation

Notation	Type	Examples in GateGuard
Solid ellipse, underlined	Key attribute	<u>MemberID</u>, <u>VisitID</u>, <u>GateID</u>
Solid ellipse	Simple attribute	Name, Email, Location, Username
Dashed ellipse	Derived attribute	Age, Duration, IsActive
Double ellipse	Multi-valued attribute	{Contacts}
Ellipse with sub-ellipses	Composite attribute	Name (FirstName, LastName)
Dashed key ellipse	Partial key (weak)	<u>GateID</u> in GateOccupancy

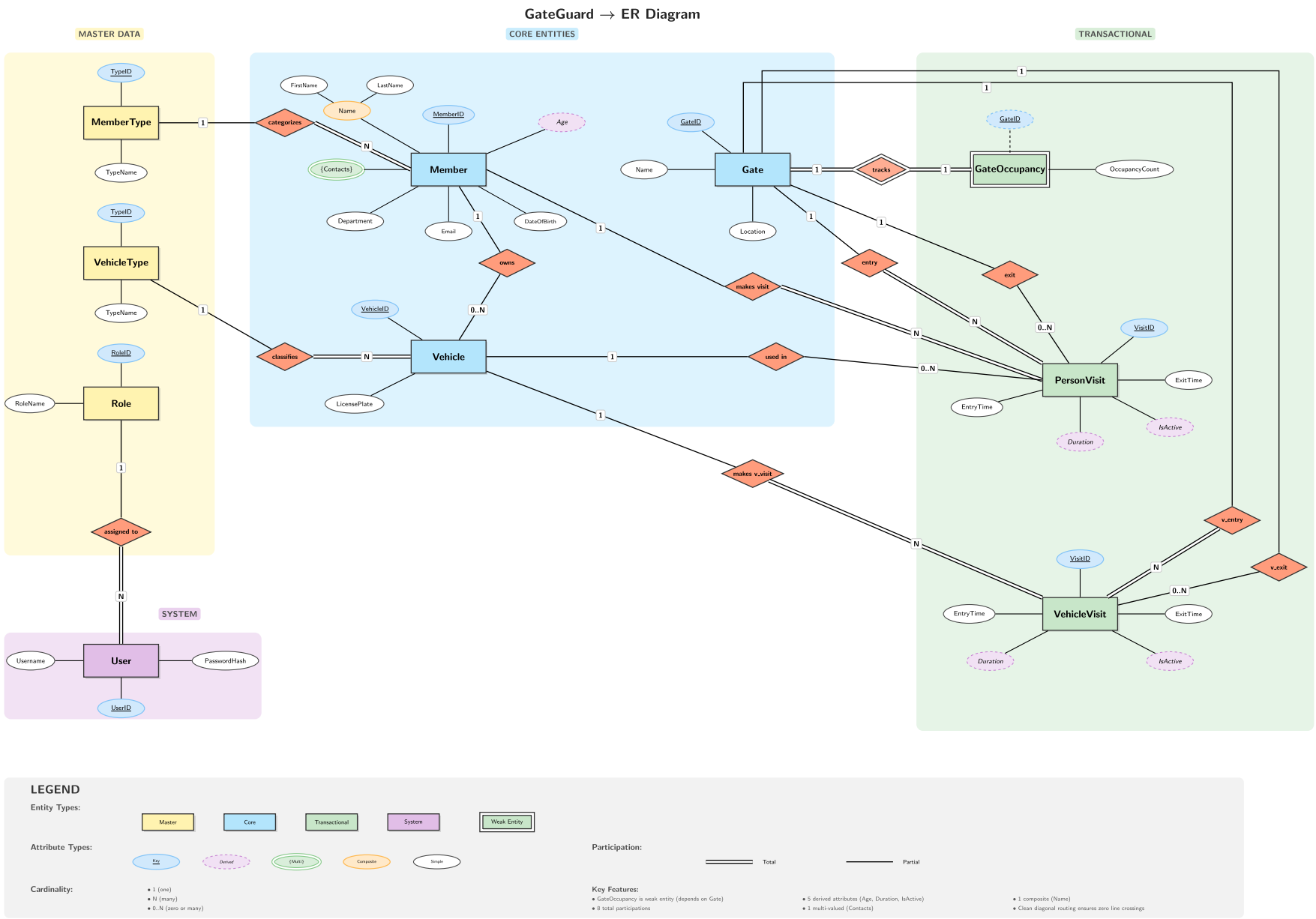

Figure 5: GateGuard – Complete ER Diagram (Crow's Foot Notation with TikZ)

3.2.3 Relationship Types

Table 7: Relationship Types and Notation

Symbol	Type	Notation	Example
Single diamond	Regular relationship	Single line	categorizes, owns
Double diamond	Identifying relationship	Double line	tracks (Gate–GateOccupancy)
Single line	Partial participation	–	Member in "owns"
Double line	Total participation	===	Member in "categorizes"

3.2.4 Weak Entity

`GateOccupancy` is modelled as a **weak entity** because:

- It cannot exist independently without its parent `Gate` entity.
- Its primary key (`GateID`) is simultaneously a foreign key referencing `Gate`.
- It is connected via an **identifying relationship** ("tracks"), represented by a double diamond.
- In the diagram, it is shown with a **double rectangle** border.
- Deletion of a Gate cascades to its GateOccupancy record (`ON DELETE CASCADE`).

4. UML to ER Transition

This section describes the systematic methodology used to convert the UML class diagram into the ER diagram, following the principles from *Elmasri & Navathe, Chapters 7 and 10* [1].

4.1 Classes to Entities Conversion

Each UML class maps directly to an ER entity. Attributes are preserved, but UML methods (behavioural elements) are *not* represented in the ER model, as ER diagrams focus exclusively on data structure.

Table 8: UML Class to ER Entity Mapping

UML Class	ER Entity	Entity Type	Conversion Notes
MEMBERTYPE	MemberType	Strong, Master	Direct mapping; 4 attributes retained
MEMBER	Member	Strong, Core	10 attributes; ER adds composite Name, derived Age
VEHICLETYPE	VehicleType	Strong, Master	Direct mapping; 4 attributes retained
VEHICLE	Vehicle	Strong, Core	6 attributes retained
GATE	Gate	Strong, Core	5 attributes retained
GATEOCCUPANCY	GateOccupancy	**Weak**	Identified by Gate via GateID
ROLE	Role	Strong, Master	Direct mapping; 4 attributes retained
USER	User	Strong, System	6 attributes retained
PERSONVISIT	PersonVisit	Strong, Transactional	9 attributes; ER adds derived Duration, IsActive
VEHICLEVISIT	VehicleVisit	Strong, Transactional	8 attributes; ER adds derived Duration, IsActive

Key Conversion Rules:

1. UML class name → ER entity name (rectangle).
2. UML private attributes (`-`) → ER attributes (ellipses).
3. UML stereotypes (`«PK»`) → ER key attributes (underlined ellipses).
4. UML stereotypes (`«FK»`) → ER relationships (diamond connectors).
5. UML methods (`+`) → *Not represented* in ER (data-only model).
6. UML constraint annotations → ER participation constraints (single/double lines).

4.2 Associations to Relationships Conversion

UML associations are converted to ER relationships with appropriate cardinality constraints:

Table 9: UML Association to ER Relationship Mapping

UML Association	UML Type	ER Relationship	Cardinality	Implementation
MemberType–Member	Composition	categorizes	1:N	TypeID FK in Member
VehicleType–Vehicle	Composition	classifies	1:N	TypeID FK in Vehicle
Member–Vehicle	Association	owns	1:N	OwnerID FK in Vehicle
Member–PersonVisit	Association	makes visit	1:N	PersonID FK in PersonVisit
Vehicle–PersonVisit	Association	used in	1:N	VehicleID FK in PersonVisit
Vehicle–VehicleVisit	Association	makes v_visit	1:N	VehicleID FK in VehicleVisit
Gate–GateOccupancy	Composition	tracks	1:1	GateID as PK+FK
Gate–PersonVisit	Association	entry	1:N	EntryGateID FK
Gate–PersonVisit	Association	exit	1:N	ExitGateID FK
Gate–VehicleVisit	Association	v_entry	1:N	EntryGateID FK
Gate–VehicleVisit	Association	v_exit	1:N	ExitGateID FK
Role–User	Association	assigned to	1:N	RoleID FK in User

Association Type Mapping Rules:

- **Composition** (♦) in UML → Strong/identifying relationship in ER, typically with `CASCADE` delete.
- **Association** (–) in UML → Standard relationship in ER, typically with `RESTRICT` delete.
- **1:M** cardinality → Foreign key placed in the "many" side entity.
- **1:1** cardinality → Foreign key placed in the dependent entity (GateOccupancy).

4.3 Key Definitions

4.3.1 Primary Keys

UML «PK» stereotypes translate directly to ER key attributes (underlined):

Table 10: Primary Key Mapping

Entity	Primary Key	Type	Notes
MemberType	TypeID	SERIAL	Auto-incrementing
Member	MemberID	SERIAL	Auto-incrementing
VehicleType	TypeID	SERIAL	Auto-incrementing
Vehicle	VehicleID	SERIAL	Auto-incrementing
Gate	GateID	SERIAL	Auto-incrementing
GateOccupancy	GateID	INTEGER	Also FK – identifying relationship
Role	RoleID	SERIAL	Auto-incrementing
User	UserID	SERIAL	Auto-incrementing
PersonVisit	VisitID	SERIAL	Auto-incrementing
VehicleVisit	VisitID	SERIAL	Auto-incrementing

4.3.2 Foreign Keys

UML «FK» stereotypes translate to ER relationships (diamonds connecting entities):

Table 11: Foreign Key Mapping (12 Relationships)

Child Table	FK Column	References	ON DELETE	ON UPDATE
Member	TypeID	MemberType(TypeID)	RESTRICT	CASCADE
Vehicle	TypeID	VehicleType(TypeID)	RESTRICT	CASCADE
Vehicle	OwnerID	Member(MemberID)	SET NULL	CASCADE
GateOccupancy	GateID	Gate(GateID)	CASCADE	CASCADE
User	RoleID	Role(RoleID)	RESTRICT	CASCADE
PersonVisit	PersonID	Member(MemberID)	RESTRICT	CASCADE
PersonVisit	EntryGateID	Gate(GateID)	RESTRICT	CASCADE
PersonVisit	ExitGateID	Gate(GateID)	RESTRICT	CASCADE
PersonVisit	VehicleID	Vehicle(VehicleID)	SET NULL	CASCADE
VehicleVisit	VehicleID	Vehicle(VehicleID)	RESTRICT	CASCADE
VehicleVisit	EntryGateID	Gate(GateID)	RESTRICT	CASCADE
VehicleVisit	ExitGateID	Gate(GateID)	RESTRICT	CASCADE

4.4 Multiplicity Adjustments

UML multiplicity notation is mapped to ER cardinality as follows:

Table 12: UML Multiplicity to ER Cardinality Mapping

UML Notation	ER Notation	Database Implementation
`1`	1	Exactly one; FK is NOT NULL
`0..1`	0..1	Zero or one; FK is nullable
`0..*`	0..N	Zero or many; FK in child table
`1..*`	1..N	One or many (min participation = 1)
`*`	N	Many (unspecified lower bound)

Concrete examples from GateGuard:

- **MemberType "1" → Member "0..*":** One type categorises many members; each member has exactly one type. Implemented via `Member.TypeID NOT NULL`.
- **Member "0..*" → Vehicle "0..1":** A member may own many vehicles; a vehicle may have zero or one owner. Implemented via `Vehicle.OwnerID` (nullable FK).
- **Gate "1" → GateOccupancy "1":** Each gate has exactly one occupancy record. Implemented via `GateOccupancy.GateID` as both PK and FK.

4.5 Participation Constraints

Table 13: Participation Constraints in the ER Diagram

Entity	Relationship	Participation	Justification
Member	categorizes	Total (double line)	Every member must have a type
Vehicle	classifies	Total (double line)	Every vehicle must have a type
PersonVisit	makes visit	Total (double line)	Every visit must have a person
VehicleVisit	makes v_visit	Total (double line)	Every vehicle visit must have a vehicle
PersonVisit	entry	Total (double line)	Every visit must have an entry gate
User	assigned to	Total (double line)	Every user must have a role
GateOccupancy	tracks	Total (double line)	Every occupancy record needs a gate
Gate	tracks	Total (double line)	Every gate must have an occupancy record
Member	owns	Partial (single line)	Not all members own vehicles
PersonVisit	used in	Partial (single line)	Not all visits involve vehicles
PersonVisit	exit	Partial (single line)	Active visits have no exit yet

5. Relationship Justifications

This section provides detailed justification for each of the **12 foreign key relationships** in the GateGuard database, including cardinality analysis, participation constraints, real-world examples from the sample data, and the SQL implementation.

5.1 R1: MemberType → Member (categorizes)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on Member side; Partial on MemberType side
FK Column	`Member.TypeID` → `MemberType.TypeID`
ON DELETE	RESTRICT (cannot delete a type if members exist)
ON UPDATE	CASCADE

Justification: One member type can categorise multiple members, but each member belongs to exactly one type. This classification is essential for access control – the system applies different entry/exit policies for Residents, Students, and Visitors.

Real-World Example from Sample Data:

- MemberType "Student" (TypeID = 2) categorises 8 members: Arjun Mehta (ID 8), Kavya Nair (ID 9), Rohan Gupta (ID 10), Divya Iyer (ID 11), Siddharth Rao (ID 12), Ananya Krishnan (ID 13), Varun Menon (ID 14), Isha Agarwal (ID 15).
- MemberType "Resident" (TypeID = 1) categorises 7 members: Dr. Rajesh Kumar (ID 1) through Dr. Manoj Joshi (ID 7).
- MemberType "Visitor" (TypeID = 3) categorises 5 members: Ramesh Kumar (ID 16) through Pradeep Nair (ID 20).

```sql
CREATE TABLE Member (
    MemberID SERIAL PRIMARY KEY,
    TypeID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_member_type FOREIGN KEY (TypeID)
        REFERENCES MemberType(TypeID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 1: R1 Implementation

5.2 R2: VehicleType → Vehicle (classifies)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on Vehicle side; Partial on VehicleType side
FK Column	`Vehicle.TypeID` → `VehicleType.TypeID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One vehicle type can classify multiple vehicles, but each vehicle has exactly one type. This categorisation is necessary for parking management, lane assignment, and security screening – different vehicle types (PrivateCar, Taxi, Bike, Truck, Bus) may have different access privileges or parking zones.

Real-World Example:

- VehicleType "PrivateCar" (TypeID = 1) classifies 10 vehicles: GJ-06-AB-1234 through GJ-06-ST-7531.
- VehicleType "Bike" (TypeID = 3) classifies 5 vehicles: GJ-06-UV-6420 through GJ-06-BB-2086.
- VehicleType "Taxi" (TypeID = 2) classifies 3 vehicles: GJ-06-CC-1975, GJ-06-DD-0864, GJ-06-EE-9753.

```sql
CREATE TABLE Vehicle (
    VehicleID SERIAL PRIMARY KEY,
    TypeID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_vehicle_type FOREIGN KEY (TypeID)
        REFERENCES VehicleType(TypeID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 2: R2 Implementation

5.3 R3: Member → Vehicle (owns)

Property	Value
Cardinality	One-to-Many (1:N), optional both sides
Participation	Partial on both sides
FK Column	`Vehicle.OwnerID` → `Member.MemberID`
ON DELETE	SET NULL (vehicle becomes unowned)
ON UPDATE	CASCADE

Justification: One member can own zero or many vehicles, and each vehicle can have zero or one owner. This relationship is optional on both sides because:

- Not all members own vehicles (visitors typically walk in).
- Some vehicles are public/commercial (taxis) and have no registered member owner.
- Members can own multiple vehicles (e.g., a faculty member with a car and a bike).

Real-World Example:

- Dr. Rajesh Kumar (MemberID = 1) owns Vehicle GJ-06-AB-1234 (VehicleID = 1).
- Arjun Mehta (MemberID = 8) owns Vehicle GJ-06-KL-2468 (VehicleID = 6).
- Vehicles GJ-06-CC-1975, GJ-06-DD-0864, GJ-06-EE-9753 (Taxis) have `OwnerID = NULL` – no member owner.
- Members Ramesh Kumar (ID 16), Sunita Devi (ID 17), etc. (Visitors) own no vehicles.

```sql
CREATE TABLE Vehicle (
    VehicleID SERIAL PRIMARY KEY,
    OwnerID INTEGER,  -- Nullable: partial participation
    CONSTRAINT fk_vehicle_owner FOREIGN KEY (OwnerID)
```

```
        REFERENCES Member(MemberID)
        ON DELETE SET NULL ON UPDATE CASCADE
);
```

Listing 3: R3 Implementation

5.4 R4: Member → PersonVisit (makes)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on PersonVisit side; Partial on Member side
FK Column	`PersonVisit.PersonID` → `Member.MemberID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One member can make multiple visits over time, but each visit record belongs to exactly one member. This is the core tracking relationship – every entry through a gate must be attributed to a specific, identified person for security and accountability.

Real-World Example:

- Arjun Mehta (MemberID = 8) made 2 visits: VisitID 1 (Jan 16, completed) and VisitID 20 (Jan 20, active – still on campus).
- Siddharth Rao (MemberID = 12) made 1 visit: VisitID 5 (Jan 16, completed).
- Ramesh Kumar (MemberID = 16, Visitor) made 2 visits: VisitID 12 (Jan 18, completed) and VisitID 25 (Jan 20, active).

```
CREATE TABLE PersonVisit (
    VisitID SERIAL PRIMARY KEY,
    PersonID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_personvisit_person FOREIGN KEY (PersonID)
        REFERENCES Member(MemberID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 4: R4 Implementation

5.5 R5: Vehicle → PersonVisit (used_in)

Property	Value
Cardinality	One-to-Many (1:N), optional
Participation	Partial on both sides
FK Column	`PersonVisit.VehicleID` → `Vehicle.VehicleID`
ON DELETE	SET NULL
ON UPDATE	CASCADE

Justification: A vehicle can be associated with multiple person visits (used at different times), and a person visit may optionally involve a vehicle. This is partial on both sides because:

- People can enter on foot, by bicycle, or as passengers (no vehicle recorded).
- Not all registered vehicles are necessarily associated with a person visit.

Real-World Example:

- PersonVisit ID 1 (Arjun Mehta, Jan 16): used Vehicle GJ-06-KL-2468 (VehicleID = 6).
- PersonVisit ID 9 (Ananya Krishnan, Jan 17): `VehicleID = NULL` – walked in.
- PersonVisit ID 20 (Arjun Mehta, Jan 20): `VehicleID = NULL` – entered without vehicle this time.

```sql
CREATE TABLE PersonVisit (
    VehicleID INTEGER,  -- Nullable: partial participation
    CONSTRAINT fk_personvisit_vehicle FOREIGN KEY (VehicleID)
        REFERENCES Vehicle(VehicleID)
        ON DELETE SET NULL ON UPDATE CASCADE
);
```

Listing 5: R5 Implementation

5.6 R6: Vehicle → VehicleVisit (makes v_visit)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on VehicleVisit side; Partial on Vehicle side
FK Column	`VehicleVisit.VehicleID` → `Vehicle.VehicleID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One vehicle can make multiple visits through the gate system, and each vehicle visit must be associated with exactly one vehicle. This provides standalone vehicle-level tracking, useful for delivery trucks, service vehicles, or when only vehicle registration is captured at the gate.

Real-World Example:

- Vehicle GJ-06-AB-1234 (VehicleID = 1) made VehicleVisit ID 1 (Jan 17, completed).
- Vehicle GJ-06-KL-2468 (VehicleID = 6) made VehicleVisit ID 4 (Jan 16, completed).
- Vehicle GJ-06-CC-1975 (VehicleID = 16, Taxi) made VehicleVisit IDs 10 (completed) and 19 (active).

```sql
CREATE TABLE VehicleVisit (
    VehicleID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_vehiclevisit_vehicle FOREIGN KEY (VehicleID)
        REFERENCES Vehicle(VehicleID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 6: R6 Implementation

5.7 R7: Gate → PersonVisit – Entry (entry)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on PersonVisit side; Partial on Gate side
FK Column	`PersonVisit.EntryGateID` → `Gate.GateID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One gate can serve as the entry point for multiple person visits, and every person visit *must* have an entry gate. This tracks which specific gate was used for entry, enabling gate-wise traffic analysis and security monitoring.

Real-World Example:

- Main Gate (GateID = 1) was entry for VisitIDs: 1, 5, 6, 12, 16, 20, 24 (7 visits).
- East Gate (GateID = 2) was entry for VisitIDs: 2, 4, 7, 10, 13, 17, 21, 25 (8 visits).
- West Gate (GateID = 3) was entry for VisitIDs: 3, 8, 11, 14, 18, 22 (6 visits).
- South Gate (GateID = 4) was entry for VisitIDs: 15, 19, 23 (3 visits).

```sql
CREATE TABLE PersonVisit (
    EntryGateID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_personvisit_entrygate FOREIGN KEY (EntryGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 7: R7 Implementation

5.8 R8: Gate → PersonVisit – Exit (exit)

Property	Value
Cardinality	One-to-Many (1:N), optional
Participation	Partial on both sides
FK Column	`PersonVisit.ExitGateID` → `Gate.GateID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One gate can serve as the exit point for multiple visits, and a person visit *may* have an exit gate (optional because the person might still be on campus). Critically, the exit gate can differ from the entry gate, supporting IIT Gandhinagar's multi-gate campus layout.

Real-World Example:

- VisitID 6 (Dr. Rajesh Kumar): Entered via Main Gate (GateID = 1), exited via **West Gate** (GateID = 3) – demonstrates multi-gate flexibility.
- VisitID 7 (Prof. Priya Sharma): Entered via East Gate (GateID = 2), exited via **South Gate** (GateID = 4).

- VisitID 20 (Arjun Mehta): `ExitGateID = NULL`, `ExitTime = NULL` – person still on campus.

```sql
CREATE TABLE PersonVisit (
    ExitGateID INTEGER,  -- Nullable: partial participation
    CONSTRAINT fk_personvisit_exitgate FOREIGN KEY (ExitGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_exitgate_if_exittime CHECK (
        (ExitTime IS NULL AND ExitGateID IS NULL) OR
        (ExitTime IS NOT NULL AND ExitGateID IS NOT NULL)
    )
);
```

Listing 8: R8 Implementation

5.9 R9: Gate → VehicleVisit – Entry (v_entry)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on VehicleVisit side; Partial on Gate side
FK Column	`VehicleVisit.EntryGateID` → `Gate.GateID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: Mirrors R7 for vehicles. One gate can be the entry point for multiple vehicle visits, and every vehicle visit must have an entry gate. This provides vehicle-specific entry tracking independent of person visits.

Real-World Example:

- Main Gate (GateID = 1) was vehicle entry for: VehicleVisit IDs 1, 4, 8, 12, 13, 15, 19 (7 visits).
- East Gate (GateID = 2) was vehicle entry for: VehicleVisit IDs 2, 5, 7, 10, 14, 20 (6 visits).

```sql
CREATE TABLE VehicleVisit (
    EntryGateID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_vehiclevisit_entrygate FOREIGN KEY (EntryGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 9: R9 Implementation

5.10 R10: Gate → VehicleVisit – Exit (v_exit)

Property	Value
Cardinality	One-to-Many (1:N), optional
Participation	Partial on both sides
FK Column	`VehicleVisit.ExitGateID` → `Gate.GateID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: Mirrors R8 for vehicles. Exit gate is optional (vehicle may still be parked on campus) and can differ from entry gate.

Real-World Example:

- VehicleVisit ID 1 (Vehicle GJ-06-AB-1234): Entered Main Gate (1), exited **West Gate** (3).
- VehicleVisit ID 2 (Vehicle GJ-06-CD-5678): Entered East Gate (2), exited **South Gate** (4).
- VehicleVisit ID 13 (Vehicle GJ-06-GH-3456): `ExitGateID = NULL` – vehicle still parked on campus.

```sql
CREATE TABLE VehicleVisit (
    ExitGateID INTEGER,  -- Nullable: partial participation
    CONSTRAINT fk_vehiclevisit_exitgate FOREIGN KEY (ExitGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_exitgate_if_exittime CHECK (
        (ExitTime IS NULL AND ExitGateID IS NULL) OR
        (ExitTime IS NOT NULL AND ExitGateID IS NOT NULL)
    )
);
```

Listing 10: R10 Implementation

5.11 R11: Gate → GateOccupancy (tracks) – Identifying

Property	Value
Cardinality	One-to-One (1:1)
Participation	Total on both sides (Identifying relationship)
FK Column	`GateOccupancy.GateID` → `Gate.GateID`
ON DELETE	CASCADE (weak entity dependency)
ON UPDATE	CASCADE

Justification: Each gate has exactly one occupancy record, and each occupancy record belongs to exactly one gate. This is an **identifying relationship** where GateOccupancy is a **weak entity** – it cannot exist without its parent Gate. The occupancy count provides real-time gate congestion monitoring.

Real-World Example:

- Main Gate (GateID = 1) ↔ GateOccupancy (GateID = 1, OccupancyCount = 5).
- East Gate (GateID = 2) ↔ GateOccupancy (GateID = 2, OccupancyCount = 3).
- West Gate (GateID = 3) ↔ GateOccupancy (GateID = 3, OccupancyCount = 8).
- South Gate (GateID = 4) ↔ GateOccupancy (GateID = 4, OccupancyCount = 2).

```sql
CREATE TABLE GateOccupancy (
    GateID INTEGER PRIMARY KEY,  -- PK + FK (identifying)
    OccupancyCount INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT fk_occupancy_gate FOREIGN KEY (GateID)
        REFERENCES Gate(GateID)
        ON DELETE CASCADE  -- Weak entity: cascade on delete
        ON UPDATE CASCADE,
    CONSTRAINT chk_occupancy_count CHECK (OccupancyCount >= 0)
);
```

Listing 11: R11 Implementation

5.12 R12: Role → User (assigned_to)

Property	Value
Cardinality	One-to-Many (1:N)
Participation	Total on User side; Partial on Role side
FK Column	`User.RoleID` → `Role.RoleID`
ON DELETE	RESTRICT
ON UPDATE	CASCADE

Justification: One role can be assigned to multiple users, but each user has exactly one role. This implements **Role-Based Access Control (RBAC)** where system permissions are determined by user role (Guard, Admin, SuperAdmin).

Real-World Example:

- Role "Guard" (RoleID = 1) is assigned to 5 users: guard1 (ID 1), guard2 (ID 2), guard3 (ID 3), guard4 (ID 4), guard5 (ID 8).
- Role "Admin" (RoleID = 2) is assigned to 2 users: admin1 (ID 5), admin2 (ID 6).
- Role "SuperAdmin" (RoleID = 3) is assigned to 1 user: superadmin (ID 7).

```sql
CREATE TABLE "User" (
    UserID SERIAL PRIMARY KEY,
    RoleID INTEGER NOT NULL,  -- Total participation
    CONSTRAINT fk_user_role FOREIGN KEY (RoleID)
        REFERENCES Role(RoleID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Listing 12: R12 Implementation

5.13 Relationship Summary

Table 14: Complete Relationship Summary (12 Foreign Keys)

#	Relationship	ER Name	Card.	DELETE	Nullable?
R1	MemberType → Member	categorizes	1:N	RESTRICT	No
R2	VehicleType → Vehicle	classifies	1:N	RESTRICT	No
R3	Member → Vehicle	owns	1:N	SET NULL	Yes
R4	Member → PersonVisit	makes visit	1:N	RESTRICT	No
R5	Vehicle → PersonVisit	used in	1:N	SET NULL	Yes
R6	Vehicle → VehicleVisit	makes v_visit	1:N	RESTRICT	No
R7	Gate → PersonVisit (entry)	entry	1:N	RESTRICT	No
R8	Gate → PersonVisit (exit)	exit	1:N	RESTRICT	Yes
R9	Gate → VehicleVisit (entry)	v_entry	1:N	RESTRICT	No
R10	Gate → VehicleVisit (exit)	v_exit	1:N	RESTRICT	Yes
R11	Gate → GateOccupancy	tracks	1:1	CASCADE	No
R12	Role → User	assigned to	1:N	RESTRICT	No

6. Additional Constraints

Beyond primary key and foreign key constraints, the GateGuard database implements comprehensive integrity constraints to enforce business rules and ensure data validity.

6.1 CHECK Constraints

6.1.1 Temporal Constraints (Visit Tables)

```sql
-- PersonVisit & VehicleVisit
CONSTRAINT chk_exittime_after_entrytime CHECK (
    ExitTime IS NULL OR ExitTime >= EntryTime
)
```

Listing 13: Exit Time Validation

Purpose: Ensures logical timing – a person or vehicle cannot exit before entering.
Valid: Entry at 09:00, Exit at 17:00 ✓ | Entry at 09:00, Exit NULL ✓ (still on campus)
Invalid: Entry at 09:00, Exit at 08:00 × (rejected by constraint)

6.1.2 Exit Gate Consistency

```sql
-- PersonVisit & VehicleVisit
CONSTRAINT chk_exitgate_if_exittime CHECK (
    (ExitTime IS NULL AND ExitGateID IS NULL) OR
    (ExitTime IS NOT NULL AND ExitGateID IS NOT NULL)
)
```

Listing 14: Exit Gate/Time Consistency

Purpose: Ensures data consistency – cannot have an exit time without an exit gate, or vice versa.
Valid States: Both NULL (on campus) or both set (exited).
Invalid: ExitTime set but ExitGateID NULL, or ExitGateID set but ExitTime NULL.

6.1.3 Occupancy Count

```sql
CONSTRAINT chk_occupancy_count CHECK (OccupancyCount >= 0)
```

Listing 15: Occupancy Count Validation

Purpose: Occupancy cannot be negative – minimum is zero (no one at gate).

6.1.4 Email Format Validation

```sql
CONSTRAINT chk_member_email CHECK (Email LIKE '%@%.%')
```

Listing 16: Email Format CHECK

Purpose: Basic email format validation ensuring the presence of '@' and '.' characters.
Valid: `arjun.mehta@iitgn.ac.in` ✓ **Invalid:** `arjunmehta` ×

6.1.5 Age Range Validation

```sql
CONSTRAINT chk_member_age CHECK (Age IS NULL OR (Age > 0 AND Age < 150)
    )
```

Listing 17: Age Range CHECK

Purpose: Ensures realistic age values when provided.
Valid: 20, 45, NULL ✓ **Invalid:** 0, −5, 200 ×

6.1.6 License Plate Format

```sql
CONSTRAINT chk_licenseplate_format CHECK (LENGTH(LicensePlate) >= 5)
```

Listing 18: License Plate Length CHECK

Purpose: Indian vehicle registration numbers require a minimum length.
Valid: `GJ-06-AB-1234` ✓ **Invalid:** `ABC` ×

6.1.7 Username and Password Constraints

```sql
CONSTRAINT chk_username_length CHECK (LENGTH(Username) >= 3)
CONSTRAINT chk_passwordhash_length CHECK (LENGTH(PasswordHash) >= 32)
```

Listing 19: User Table Constraints

Purpose: Ensure meaningful usernames (min 3 chars) and properly hashed passwords (min 32 chars for MD5).

6.1.8 Type Name Enumeration Constraints

```sql
-- MemberType
CONSTRAINT chk_membertype_name CHECK (
    TypeName IN ('Resident', 'Student', 'Visitor'))
-- VehicleType
CONSTRAINT chk_vehicletype_name CHECK (
    TypeName IN ('PrivateCar', 'Taxi', 'Bike', 'Truck', 'Bus'))
-- Role
CONSTRAINT chk_role_name CHECK (
    RoleName IN ('Guard', 'Admin', 'SuperAdmin'))
```

Listing 20: Enumeration CHECK Constraints

Purpose: Enforce controlled vocabulary – only predefined type/role values are allowed.

6.2 UNIQUE Constraints

Table 15: UNIQUE Constraints

Table	Column	Purpose
Member	Email	Each member has a unique email for identification
Vehicle	LicensePlate	Each vehicle has a unique registration number
User	Username	Each system user has a unique login identifier
MemberType	TypeName	Prevent duplicate member type definitions
VehicleType	TypeName	Prevent duplicate vehicle type definitions
Role	RoleName	Prevent duplicate role definitions

6.3 NOT NULL Constraints

Every table meets the minimum requirement of **at least 3 NOT NULL columns**:

Table 16: NOT NULL Column Counts per Table

Table	NN Count	NOT NULL Columns
MemberType	3	TypeID, TypeName, CreatedAt
Member	7	MemberID, Name, Email, ContactNumber, TypeID, CreatedAt, UpdatedAt
VehicleType	3	TypeID, TypeName, CreatedAt
Vehicle	5	VehicleID, LicensePlate, TypeID, CreatedAt, UpdatedAt
Gate	4	GateID, Name, Location, CreatedAt
GateOccupancy	3	GateID, OccupancyCount, UpdatedAt
Role	3	RoleID, RoleName, CreatedAt
User	6	UserID, Username, PasswordHash, RoleID, CreatedAt, UpdatedAt
PersonVisit	5	VisitID, PersonID, EntryGateID, EntryTime, CreatedAt
VehicleVisit	5	VisitID, VehicleID, EntryGateID, EntryTime, CreatedAt

6.4 Referential Integrity Summary

Table 17: Referential Integrity Actions

Child Table	FK	ON DELETE	ON UPDATE	Rationale
Member	TypeID	RESTRICT	CASCADE	Cannot delete type with members
Vehicle	TypeID	RESTRICT	CASCADE	Cannot delete type with vehicles
Vehicle	OwnerID	SET NULL	CASCADE	Vehicle becomes unowned
GateOccupancy	GateID	CASCADE	CASCADE	Weak entity: delete with gate
User	RoleID	RESTRICT	CASCADE	Cannot delete role with users
PersonVisit	PersonID	RESTRICT	CASCADE	Cannot delete member with visits
PersonVisit	EntryGateID	RESTRICT	CASCADE	Cannot delete gate with visits
PersonVisit	ExitGateID	RESTRICT	CASCADE	Cannot delete gate with visits
PersonVisit	VehicleID	SET NULL	CASCADE	Visit record preserved
VehicleVisit	VehicleID	RESTRICT	CASCADE	Cannot delete vehicle with visits
VehicleVisit	EntryGateID	RESTRICT	CASCADE	Cannot delete gate with visits
VehicleVisit	ExitGateID	RESTRICT	CASCADE	Cannot delete gate with visits

6.5 Indexes

The database defines **22 indexes** for query performance optimisation:

Table 18: Database Indexes

Table	Index	Purpose
MemberType	idx_membertype_name	Fast lookup by type name
Member	idx_member_type	Fast join on TypeID
Member	idx_member_email	Fast email search (also UNIQUE)
Member	idx_member_name	Fast name-based queries
VehicleType	idx_vehicletype_name	Fast lookup by type name
Vehicle	idx_vehicle_type	Fast join on TypeID
Vehicle	idx_vehicle_owner	Fast join on OwnerID
Vehicle	idx_vehicle_licenseplate	Fast plate lookup (UNIQUE)
Gate	idx_gate_name, idx_gate_location	Fast gate identification
GateOccupancy	idx_occupancy_count	Occupancy reporting queries
Role	idx_role_name	Fast role lookup
User	idx_user_role, idx_user_username	Fast auth and role joins
PersonVisit	7 indexes	PersonID, gates, vehicle, timestamps, active visits
VehicleVisit	6 indexes	VehicleID, gates, timestamps, active visits

Partial Index (advanced):

```sql
CREATE INDEX idx_personvisit_active
    ON PersonVisit(PersonID, ExitTime) WHERE ExitTime IS NULL;
CREATE INDEX idx_vehiclevisit_active
    ON VehicleVisit(VehicleID, ExitTime) WHERE ExitTime IS NULL;
```

Listing 21: Partial Index for Active Visits

These partial indexes significantly speed up queries for currently active visits (people/vehicles still on campus).

7. Database Schema Overview

This section provides a consolidated view of the complete database schema as implemented in Module A.

7.1 Schema Statistics

Table 19: GateGuard Database Statistics

Metric	Value
Total Tables	10
Total Foreign Key Relationships	12
Total CHECK Constraints	12
Total UNIQUE Constraints	6
Total Indexes	22
Total Sample Data Records	105
Normalization Level	3NF
Database Engine	PostgreSQL 12+

7.2 Sample Data Distribution

Table 20: Sample Data Record Counts

Table	Rows	Data Description
MemberType	3	Resident, Student, Visitor
Member	20	7 Residents, 8 Students, 5 Visitors
VehicleType	5	PrivateCar, Taxi, Bike, Truck, Bus
Vehicle	18	10 Cars, 5 Bikes, 3 Taxis
Gate	4	Main, East, West, South gates
GateOccupancy	4	One record per gate
Role	3	Guard, Admin, SuperAdmin
User	8	5 Guards, 2 Admins, 1 SuperAdmin
PersonVisit	25	15 completed, 10 active
VehicleVisit	20	12 completed, 8 active
Total	**105**	

7.3 Normalization Analysis

The GateGuard schema satisfies **Third Normal Form (3NF)**:

- **1NF:** All attributes are atomic. No repeating groups or multi-valued attributes in the relational schema. Each cell contains a single value.
- **2NF:** All non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist (all tables use single-column primary keys).

- **3NF:** No transitive dependencies. Non-key attributes depend only on the primary key, not on other non-key attributes. Type names are stored in separate lookup tables (MemberType, VehicleType, Role) rather than directly in entity tables.

Design Decisions for Normalization:

- **`MemberType`**, **`VehicleType`**, and **`Role`** are extracted as separate lookup tables to eliminate transitive dependencies and enable controlled vocabulary enforcement via CHECK constraints.
- **`GateOccupancy`** is separated from **`Gate`** to isolate frequently-updated occupancy counts from relatively static gate information, reducing write contention.
- **`PersonVisit`** and **`VehicleVisit`** are kept as separate tables (rather than a single "Visit" table) to avoid NULL-heavy rows and to clearly model the different attributes and constraints of person visits vs. vehicle visits.

8. Team Member Contributions

Table 21: Team Member Contributions

Name	Roll No.	Contributions
Pushkar Nikunjkumar Modi	25120035	Database schema design (10 tables, constraints, indexes); SQL dump creation and testing; Sequence diagram creation(TikZ/LaTeX); Module B report compilation; UML–ER alignment verification
Chintan Bhikhabhai Akhaja	25120027	ER diagram design using TikZ/LaTeX; Entity-relationship modelling; Attribute type notation (derived, composite, multi-valued); Relationship cardinality verification
Krina Paghadar	24110169	UML class diagram design (draw.io); Use case diagram (draw.io); Class attribute and method specification; Multiplicity and association labelling
Divisht	24110112	Use case diagram design in Notebook; Activity diagram creation in Notebook; Workflow modelling (entry/exit flows); System actor identification
Akshata Pawar	25120026	Sample data generation (105 records); Data validation and integrity testing; Seed SQL script creation; Constraint violation testing

8.1 Collaborative Efforts

The following tasks were completed collaboratively by all team members:

- System requirements analysis and scope definition.
- Database architecture discussions and entity identification.
- Normalization review and 3NF verification.
- Cross-verification of UML diagram ↔ ER diagram ↔ SQL schema alignment.
- Peer review of all diagrams and documentation.
- Testing of the SQL dump file on a fresh PostgreSQL database.
- Final quality assurance and submission preparation.

8.2 Tools and Technologies

Table 22: Tools and Technologies Used

Category	Tool
Database	PostgreSQL 12+
SQL Development	pgAdmin 4, psql CLI
UML Diagrams	PlantUML, Visual Studio Code
ER Diagram	LaTeX with TikZ/PGF
Sequence/Activity Diagrams	Mermaid.js, PlantUML
Version Control	Git, GitHub
Collaboration	VS Code Live Share, Discord

9. Conclusion

This report presents the complete conceptual design of the **GateGuard Entry Gate Management System** for IIT Gandhinagar, developed as part of CS432 Track-1 Assignment 1, Module B.

9.1 Summary of Deliverables

1. **UML Diagrams:** Four UML diagrams were created – Class Diagram (showing 10 classes with attributes, methods, and 10 associations), Sequence Diagram (modelling entry/exit workflows), Use Case Diagram (capturing actor–system interactions), and Activity Diagram (modelling decision points and parallel operations).
2. **ER Diagram:** A comprehensive Entity-Relationship diagram was designed using Ti*k*Z/LaTeX, featuring 10 entities (including 1 weak entity), 12 relationships, colour-coded functional regions, and standard ER notation for key attributes, derived attributes, composite attributes, multi-valued attributes, and participation constraints.
3. **UML-to-ER Transition:** A detailed systematic mapping was documented, showing how UML classes convert to ER entities, UML associations convert to ER relationships, stereotypes map to keys, and multiplicity translates to cardinality.
4. **Relationship Justifications:** All 12 foreign key relationships were individually justified with cardinality analysis, participation constraints, real-world examples from the 105-record sample dataset, and the actual SQL implementation code.
5. **Constraint Documentation:** All 30+ constraints (12 CHECK, 6 UNIQUE, 12 FK, and NOT NULL constraints across all tables) were documented with purpose, examples, and valid/invalid cases.

9.2 Key Design Strengths

- **3NF Compliance:** Eliminates all transitive dependencies through proper table decomposition.
- **Multi-Gate Support:** Entry and exit gates can differ, modelling real campus scenarios.
- **Weak Entity Modelling:** GateOccupancy correctly modelled as a weak entity with identifying relationship.
- **Dual Visit Tracking:** Separate PersonVisit and VehicleVisit tables for clean, flexible tracking.
- **Comprehensive Constraints:** Business rules enforced at the database level, not just the application level.
- **Performance Indexes:** 22 indexes including partial indexes for active visit queries.
- **105 Sample Records:** Realistic data covering completed visits, active visits, multi-gate exits, pedestrian entries, and unowned vehicles.

References

[1] Elmasri, R. & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson Education. – Chapters 7 (ER Modelling) and 10 (UML/Object Modelling).

[2] Silberschatz, A., Korth, H. F. & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education. – Chapter 6 (Database Design Using the E-R Model).

[3] Connolly, T. & Begg, C. (2014). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson Education.

[4] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/

[5] Object Management Group (OMG). (2015). *OMG Unified Modeling Language (UML) Version 2.5*. https://www.omg.org/spec/UML/2.5/

[6] PlantUML. (2024). *PlantUML Class Diagram Syntax*. https://plantuml.com/class-diagram

[7] Tantau, T. (2024). *The TikZ and PGF Packages – Manual for version 3.1.10*. https://tikz.dev/

[8] CS432 Database Management Systems – Lecture Notes and Slides. IIT Gandhinagar, Spring 2026.

A. Complete SQL Schema

```sql
-- 1. MemberType
CREATE TABLE MemberType (
    TypeID SERIAL PRIMARY KEY,
    TypeName VARCHAR(50) NOT NULL UNIQUE,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_membertype_name
        CHECK (TypeName IN ('Resident', 'Student', 'Visitor'))
);

-- 2. Member
CREATE TABLE Member (
    MemberID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(255) NOT NULL UNIQUE,
    ContactNumber VARCHAR(20) NOT NULL,
    Image BYTEA,
    Age INTEGER,
    Department VARCHAR(100),
    TypeID INTEGER NOT NULL,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_member_type FOREIGN KEY (TypeID)
        REFERENCES MemberType(TypeID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_member_email CHECK (Email LIKE '%@%.%'),
    CONSTRAINT chk_member_age
        CHECK (Age IS NULL OR (Age > 0 AND Age < 150))
);

-- 3. VehicleType
CREATE TABLE VehicleType (
    TypeID SERIAL PRIMARY KEY,
    TypeName VARCHAR(50) NOT NULL UNIQUE,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_vehicletype_name
        CHECK (TypeName IN ('PrivateCar','Taxi','Bike','Truck','Bus'))
);

-- 4. Vehicle
CREATE TABLE Vehicle (
    VehicleID SERIAL PRIMARY KEY,
    LicensePlate VARCHAR(20) NOT NULL UNIQUE,
    TypeID INTEGER NOT NULL,
    OwnerID INTEGER,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vehicle_type FOREIGN KEY (TypeID)
        REFERENCES VehicleType(TypeID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_vehicle_owner FOREIGN KEY (OwnerID)
        REFERENCES Member(MemberID)
        ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_licenseplate_format
        CHECK (LENGTH(LicensePlate) >= 5)
```

```sql
);

-- 5. Gate
CREATE TABLE Gate (
    GateID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Location VARCHAR(255) NOT NULL,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_gate_name CHECK (LENGTH(Name) > 0)
);

-- 6. GateOccupancy (Weak Entity)
CREATE TABLE GateOccupancy (
    GateID INTEGER PRIMARY KEY,
    OccupancyCount INTEGER NOT NULL DEFAULT 0,
    UpdatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_occupancy_gate FOREIGN KEY (GateID)
        REFERENCES Gate(GateID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_occupancy_count
        CHECK (OccupancyCount >= 0)
);

-- 7. Role
CREATE TABLE Role (
    RoleID SERIAL PRIMARY KEY,
    RoleName VARCHAR(50) NOT NULL UNIQUE,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_role_name
        CHECK (RoleName IN ('Guard', 'Admin', 'SuperAdmin'))
);

-- 8. User
CREATE TABLE "User" (
    UserID SERIAL PRIMARY KEY,
    Username VARCHAR(50) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    RoleID INTEGER NOT NULL,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_user_role FOREIGN KEY (RoleID)
        REFERENCES Role(RoleID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_username_length
        CHECK (LENGTH(Username) >= 3),
    CONSTRAINT chk_passwordhash_length
        CHECK (LENGTH(PasswordHash) >= 32)
);

-- 9. PersonVisit
CREATE TABLE PersonVisit (
    VisitID SERIAL PRIMARY KEY,
    PersonID INTEGER NOT NULL,
    EntryGateID INTEGER NOT NULL,
    EntryTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ExitGateID INTEGER,
```

```sql
    ExitTime TIMESTAMP,
    VehicleID INTEGER,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_personvisit_person FOREIGN KEY (PersonID)
        REFERENCES Member(MemberID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_personvisit_entrygate
        FOREIGN KEY (EntryGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_personvisit_exitgate
        FOREIGN KEY (ExitGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_personvisit_vehicle
        FOREIGN KEY (VehicleID)
        REFERENCES Vehicle(VehicleID)
        ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_exittime_after_entrytime
        CHECK (ExitTime IS NULL OR ExitTime >= EntryTime),
    CONSTRAINT chk_exitgate_if_exittime CHECK (
        (ExitTime IS NULL AND ExitGateID IS NULL) OR
        (ExitTime IS NOT NULL AND ExitGateID IS NOT NULL))
);

-- 10. VehicleVisit
CREATE TABLE VehicleVisit (
    VisitID SERIAL PRIMARY KEY,
    VehicleID INTEGER NOT NULL,
    EntryGateID INTEGER NOT NULL,
    EntryTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ExitGateID INTEGER,
    ExitTime TIMESTAMP,
    CreatedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vehiclevisit_vehicle
        FOREIGN KEY (VehicleID)
        REFERENCES Vehicle(VehicleID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_vehiclevisit_entrygate
        FOREIGN KEY (EntryGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_vehiclevisit_exitgate
        FOREIGN KEY (ExitGateID)
        REFERENCES Gate(GateID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_exittime_after_entrytime
        CHECK (ExitTime IS NULL OR ExitTime >= EntryTime),
    CONSTRAINT chk_exitgate_if_exittime CHECK (
        (ExitTime IS NULL AND ExitGateID IS NULL) OR
        (ExitTime IS NOT NULL AND ExitGateID IS NOT NULL))
);
```

Listing 22: GateGuard Complete Schema (Module A)